

YOLO + Zonas Delimitadas + Cámara en Tiempo Real

Guía práctica de implementación de Computer Vision

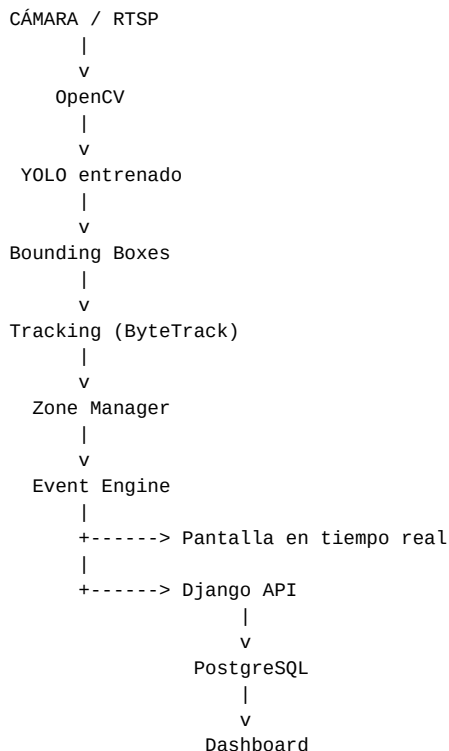
Detección de objetos → Bounding Boxes → Tracking → Zonas poligonales → Eventos → Django/PostgreSQL

Objetivo: convertir un modelo YOLO ya entrenado en un sistema capaz de procesar una cámara en tiempo real, identificar objetos, saber en qué zona se encuentran y registrar eventos de entrada, salida y permanencia.

Componente	Tecnología recomendada
Detección	YOLO / Ultralytics
Video	OpenCV
Tracking	ByteTrack
Zonas	Polígonos + OpenCV
Backend	Django / Django REST Framework
Base de datos	PostgreSQL
Frontend	Dashboard web (opcional)

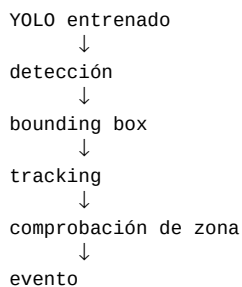
1. Arquitectura general

La cámara produce frames. OpenCV los recibe, YOLO detecta objetos, el tracker mantiene sus IDs y el módulo de zonas determina si cada objeto está dentro o fuera de una región definida.



2. ¿Es necesario volver a entrenar YOLO?

No necesariamente. Si el modelo ya detecta correctamente las clases que necesitas, las zonas son una capa de lógica posterior a la detección. El flujo es:



Solo necesitas entrenar o reentrenar YOLO si quieres reconocer clases que el modelo actual no detecta bien o si su precisión es insuficiente en tu escenario.

3. Estructura recomendada del proyecto



```
■ ■■■ events.py
■
■■■ requirements.txt
■■■ README.md
```

4. Instalación

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux/macOS
source .venv/bin/activate

pip install ultralytics opencv-python numpy
```

Si posteriormente se integra Django:

```
pip install django djangorestframework psycopg2-binary
```

5. Probar el modelo entrenado con una imagen

```
from ultralytics import YOLO

model = YOLO("models/best.pt")

results = model("images/referencia.jpg")

for result in results:
    result.show()
```

Antes de pasar a tiempo real, conviene comprobar que el modelo reconoce correctamente las clases y produce bounding boxes adecuados.

6. Conectar una webcam

```
import cv2
from ultralytics import YOLO

model = YOLO("models/best.pt")

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    results = model(frame)

    annotated = results[0].plot()

    cv2.imshow("YOLO - Camara", annotated)

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

La tecla ESC finaliza la aplicación. El número 0 corresponde normalmente a la webcam principal; puede cambiarse a 1, 2, etc. si existen varias cámaras.

7. Utilizar una cámara IP

Muchas cámaras de vigilancia entregan video mediante RTSP. En ese caso se reemplaza el índice de la webcam por la dirección del stream.

```
cap = cv2.VideoCapture(
    "rtsp://usuario:password@IP:554/stream"
)
```

La dirección exacta depende del fabricante y de la configuración de la cámara. Para producción conviene proteger las credenciales y no escribirlas directamente en el código.

8. Definir zonas poligonales

Una zona puede representarse mediante una lista de puntos. Los puntos están expresados en coordenadas de píxeles de la imagen.

```
zona_entrada = [
    (100, 100),
    (500, 100),
    (500, 400),
    (100, 400)
]

zona_restringida = [
    (600, 150),
    (900, 150),
    (850, 450),
    (600, 400)
]
```

8.1 Guardar las zonas en JSON

```
{
  "zonas": [
    {
      "nombre": "Entrada",
      "puntos": [
        [100, 100],
        [500, 100],
        [500, 400],
        [100, 400]
      ]
    },
    {
      "nombre": "Zona restringida",
      "puntos": [
        [600, 150],
        [900, 150],
        [850, 450],
        [600, 400]
      ]
    }
  ]
}
```

8.2 Cargar las zonas

```
import json

with open("config/zonas.json", "r", encoding="utf-8") as f:
    config = json.load(f)

for zona in config["zonas"]:
    print(zona["nombre"])
    print(zona["puntos"])
```

9. Dibujar las zonas sobre el frame

```
import cv2
import numpy as np

def dibujar_zona(frame, puntos, nombre):
    pts = np.array(puntos, dtype=np.int32)

    cv2.polylines(
        frame,
        [pts],
```

```

        True,
        (0, 255, 0),
        2
    )

    x, y = pts[0]
    cv2.putText(
        frame,
        nombre,
        (int(x), int(y) - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 0),
        2
    )

```

10. Determinar si un objeto está dentro de una zona

OpenCV dispone de `pointPolygonTest`. Una estrategia simple es utilizar el centro del bounding box.

```

import cv2
import numpy as np

def punto_dentro_zona(x, y, puntos):
    polygon = np.array(puntos, dtype=np.int32)

    resultado = cv2.pointPolygonTest(
        polygon,
        (float(x), float(y)),
        False
    )

    return resultado >= 0

```

10.1 Centro del bounding box

```

cx = (x1 + x2) / 2
cy = (y1 + y2) / 2

dentro = punto_dentro_zona(cx, cy, zona)

```

10.2 Punto inferior central para personas

En escenarios donde importa el lugar donde la persona pisa el suelo, suele ser más útil el punto inferior central del bounding box.

```

px = (x1 + x2) / 2
py = y2

dentro = punto_dentro_zona(px, py, zona)

```

Esta decisión depende del problema. Para objetos suspendidos, vehículos u otros casos puede ser preferible el centro u otra referencia geométrica.

11. Tracking con ByteTrack

Para análisis temporal necesitamos saber que el objeto detectado en un frame corresponde al mismo objeto observado en los siguientes frames. Ultralytics permite utilizar tracking.

```

from ultralytics import YOLO

model = YOLO("models/best.pt")

results = model.track(
    frame,
    persist=True,
    tracker="bytetrack.yaml"
)

```

Con tracking podemos obtener IDs, por ejemplo:

```

Persona ID 7

```

12. Flujo completo en tiempo real

```
import cv2
import json
import numpy as np
from ultralytics import YOLO

MODEL_PATH = "models/best.pt"
ZONES_PATH = "config/zonas.json"

model = YOLO(MODEL_PATH)

with open(ZONES_PATH, "r", encoding="utf-8") as f:
    config = json.load(f)

zonas = config["zonas"]

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    results = model.track(
        frame,
        persist=True,
        tracker="bytetrack.yaml",
        verbose=False
    )

    annotated = frame.copy()

    # Dibujar zonas
    for zona in zonas:
        puntos = np.array(zona["puntos"], dtype=np.int32)

        cv2.polylines(
            annotated,
            [puntos],
            True,
            (0, 255, 0),
            2
        )

        x, y = puntos[0]

        cv2.putText(
            annotated,
            zona["nombre"],
            (int(x), int(y) - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (0, 255, 0),
            2
        )

    # Dibujar detecciones
    annotated = results[0].plot(img=annotated)

    cv2.imshow("Sistema", annotated)

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Este ejemplo muestra la arquitectura base. El siguiente paso es extraer las cajas y sus IDs para ejecutar la lógica de zonas y eventos.

13. Detectar entrada y salida de una zona

Para cada ID conviene mantener un estado. Por ejemplo, si una persona estaba fuera y ahora está dentro, se genera un evento de entrada.

```
estado = {
  7: {
    "zona": "Entrada",
    "dentro": True
  }
}
```

Lógica conceptual:

```

  ANTES          AHORA
fuera      →     dentro
              ↓
            EVENTO ENTRADA

  ANTES          AHORA
dentro     →     fuera
              ↓
            EVENTO SALIDA
```

13.1 Estado por objeto y zona

```
estados = {
  # object_id: {zone_name: bool}
}
```

En una implementación más completa, el estado puede incluir timestamp de entrada, último frame visto, clase, confianza y posición.

14. Calcular tiempo de permanencia

```
from datetime import datetime

entradas = {}

# Cuando entra:
entradas[(object_id, zone_name)] = datetime.now()

# Cuando sale:
inicio = entradas.pop((object_id, zone_name), None)

if inicio:
    duracion = (datetime.now() - inicio).total_seconds()
    print(f"Tiempo: {duracion:.2f} segundos")
```

En producción es mejor utilizar timestamps consistentes y manejar pérdidas temporales del tracking para evitar considerar una detección perdida como una salida inmediata.

15. Registrar eventos

Un evento puede tener una estructura como:

```
{
  "camera_id": 1,
  "object_id": 27,
  "class": "person",
  "zone": "zona_restringida",
  "event": "entry",
  "confidence": 0.94,
  "timestamp": "2026-08-11T16:30:25"
}
```

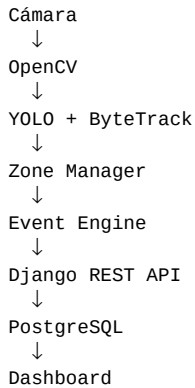
Eventos posibles:

- entry: objeto entra en una zona.
- exit: objeto sale de una zona.

- dwell: se supera un tiempo de permanencia.
- line_crossing: se cruza una línea virtual.
- alert: se cumple una condición de riesgo o restricción.

16. Integración con Django y PostgreSQL

Una arquitectura completa puede separar el procesamiento de video del backend web. El proceso de Computer Vision analiza el stream y Django expone los datos mediante una API.



16.1 Ejemplo de modelo Django

```

from django.db import models

class Camera(models.Model):
    name = models.CharField(max_length=100)
    stream_url = models.TextField()

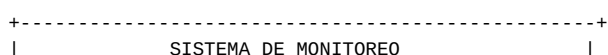
class Zone(models.Model):
    camera = models.ForeignKey(Camera, on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    points = models.JSONField()

class DetectionEvent(models.Model):
    camera = models.ForeignKey(Camera, on_delete=models.CASCADE)
    object_id = models.IntegerField()
    object_class = models.CharField(max_length=100)
    zone = models.ForeignKey(Zone, null=True, on_delete=models.SET_NULL)
    event_type = models.CharField(max_length=50)
    confidence = models.FloatField(null=True)
    timestamp = models.DateTimeField(auto_now_add=True)
  
```

17. Dashboard

Un dashboard puede mostrar:

- Video de la cámara.
- Bounding boxes.
- Nombre y límites de las zonas.
- IDs de tracking.
- Cantidad de objetos por zona.
- Eventos recientes.
- Tiempo de permanencia.
- Alertas.
- Historial por cámara.



CAMARA 1		EVENTOS
YOLO + ZONAS		Persona 7 - ENTRA Persona 4 - SALE Auto 3 - ENTRA
Zona Entrada:	5 objetos	
Zona Riesgo:	1 objeto	
Alertas:	2	

18. Varias cámaras

Cuando existen varias cámaras, cada cámara debe tener su propia configuración de zonas, porque las coordenadas dependen de la perspectiva.

Cámara 1
 ■■■ modelo
 ■■■ stream
 ■■■ zonas

Cámara 2
 ■■■ modelo
 ■■■ stream
 ■■■ zonas

Cámara 3
 ■■■ modelo
 ■■■ stream
 ■■■ zonas

No conviene reutilizar directamente las coordenadas de una cámara en otra si cambia la posición o resolución.

19. Editor de zonas recomendado

Para un proyecto más profesional, conviene construir un pequeño editor que permita configurar las zonas sin modificar código.

- Seleccionar una cámara.
- Mostrar un frame de referencia.
- Crear una nueva zona.
- Hacer clic para agregar vértices.
- Cerrar el polígono.
- Asignar nombre y tipo de zona.
- Editar o eliminar zonas.
- Guardar la configuración.
- Recargar las zonas sin reiniciar todo el sistema.

Una configuración podría ser:

```
{
  "camera_id": 1,
  "zonas": [
    {
      "id": 1,
      "nombre": "Entrada",
      "tipo": "permitida",
      "puntos": [[100,100], [500,100], [500,400], [100,400]]
    },
    {
```

```

    "id": 2,
    "nombre": "Zona restringida",
    "tipo": "restringida",
    "puntos": [[600,150], [900,150], [850,450], [600,400]]
  }
]
}

```

20. Rendimiento en tiempo real

La velocidad dependerá del modelo, resolución, hardware, cantidad de cámaras y número de objetos. Para mejorar rendimiento:

- Usar un modelo YOLO adecuado al hardware.
- Reducir la resolución de entrada si la precisión lo permite.
- Usar GPU cuando esté disponible.
- Evitar procesar frames innecesarios.
- Utilizar tracking para reducir trabajo adicional.
- Separar captura, inferencia y envío de eventos cuando sea necesario.
- Medir FPS y latencia.

20.1 Métricas útiles

```

FPS
Latencia por frame
Tiempo de inferencia YOLO
Número de objetos
Número de cámaras
CPU
GPU
Memoria
Frames perdidos

```

21. Casos de uso

- Control de acceso: detectar personas que entran a un área.
- Estacionamientos: determinar ocupación de plazas.
- Seguridad: detectar presencia en zonas restringidas.
- Retail: analizar permanencia en áreas de una tienda.
- Industria: monitorear áreas peligrosas.
- Logística: controlar vehículos o personas en sectores definidos.
- Deportes: analizar presencia y movimiento dentro de una cancha.
- Conteo de entradas y salidas.

22. Incorporar Machine Learning adicional

YOLO ya es Deep Learning. Sin embargo, puedes utilizar los datos generados por YOLO + tracking + zonas como variables para un segundo modelo.

```

YOLO
|
+-- clase
+-- confianza
+-- x
+-- y
+-- ancho
+-- alto

```

```

|
+-- Tracking
|   +-- ID
|   +-- velocidad
|   +-- dirección
|
+-- Zonas
    +-- zona
    +-- tiempo
    +-- entradas
    +-- salidas
        |
        v
        Feature Engineering
        |
        v
        XGBoost / Random Forest
        |
        v
        Clasificación / Riesgo

```

Ejemplo de variables:

```

object_class
confidence
zone_id
time_in_zone
speed
direction
number_of_zone_entries
number_of_zone_exits
bounding_box_area
distance_travelled

```

23. Ejemplo de evolución del proyecto

```

FASE 1
Imagen + YOLO
↓
Bounding boxes

FASE 2
Video + YOLO
↓
Detección en tiempo real

FASE 3
Video + YOLO + zonas
↓
Objetos dentro/fuera

FASE 4
YOLO + ByteTrack
↓
IDs persistentes

FASE 5
Tracking + zonas
↓
Entrada / salida / permanencia

FASE 6
Eventos
↓
PostgreSQL

FASE 7
Django REST API
↓
Dashboard

FASE 8
Features
↓
Machine Learning adicional

```

24. Recomendación de implementación

Para desarrollar el proyecto de forma ordenada, se recomienda no comenzar inmediatamente con Django ni con varias cámaras.

- Primero comprobar el modelo YOLO con fotografías reales.
- Después procesar una webcam.
- Implementar el editor de zonas.
- Guardar y cargar zonas desde JSON.
- Agregar detección dentro/fuera de cada zona.
- Agregar ByteTrack.
- Implementar eventos de entrada y salida.
- Medir tiempo de permanencia.
- Persistir eventos en PostgreSQL.
- Crear API Django.
- Finalmente crear dashboard y soporte para varias cámaras.

25. Checklist de implementación

Elemento	Estado esperado
Modelo YOLO entrenado	best.pt disponible
Webcam	Video recibido
Cámara IP	RTSP funcional
Bounding boxes	Detecciones visibles
Tracking	IDs persistentes
Editor de zonas	Polígonos configurables
Persistencia de zonas	JSON o BD
Detección de zona	Dentro/fuera
Eventos	Entrada/salida
Permanencia	Segundos calculados
PostgreSQL	Eventos almacenados
Django API	Datos consultables
Dashboard	Visualización

26. Consideraciones finales

La clave conceptual es separar claramente las responsabilidades:

- YOLO: detecta qué objetos aparecen y dónde están.
- Tracking: mantiene la identidad de los objetos entre frames.
- Zonas: determina en qué región está cada objeto.
- Event Engine: transforma cambios de estado en eventos.

- Django: expone y administra los datos.
- PostgreSQL: almacena la información histórica.
- Dashboard: permite visualizar el sistema.

Con esta arquitectura puedes comenzar con una sola cámara y posteriormente escalar a cámaras IP, múltiples zonas, múltiples cámaras, alertas, analítica histórica y modelos de Machine Learning sobre el comportamiento observado.

Nota: Este documento es una guía de implementación. Los parámetros de cámara, resolución, modelo YOLO, tracker y arquitectura final deben ajustarse al hardware y al escenario real.